

A Model of Agile Evolution and Maintenance Process

Mira Kajko-Mattsson and Jaana Nyfjorð
 Department of Computer and Systems Sciences
mira@dsv.su.se, jaana@dsv.su.se

Abstract

Most of the agile methods mainly concentrate on the development phase. None of them however is explicitly dedicated to the evolution and maintenance domain. In this paper, we outline an agile evolution and maintenance process model and evaluate it within two Canadian software organizations. We do this by comparing current agile development process models to the industrial evolution and maintenance practice in order to find issues relevant for the evolution and maintenance domain. Our results show that some changes need be made in the current agile methods.

1. Introduction

Agile processes have been extensively studied in both academia and industry. Most of the current research has mainly concentrated on the development phase. Little has been done to adapt them to the evolution and maintenance domain. So far, we have not found any publications that approach agile evolution and maintenance whatsoever.

In this paper, we outline an agile evolution and maintenance process model and evaluate it within two Canadian organizations. Our goal is to find out how agility is approached in the industrial evolution and maintenance today.

The remainder of this paper is as follows. Section 2 presents our research method. Section 3 describes the evolution and maintenance agile model. Section 4 lists the differences between our model and the industrial agile evolution and maintenance practice. Finally, conclusions and future research are presented in Section 5.

2. Research method

In this section, our research method is described. The research phases are presented in Sections 2.1. Section 2.2 describes the two Canadian organizations studied.

2.1 Research phases

As a first step, we studied current agile models and elicited activities belonging to both the pre-implementation and implementation phases on business and engineering levels. We then put these into a synthesized *Evolution and Maintenance Process Model*.

There is a number of existing agile process models. To avoid bias towards a specific process model and to achieve objective representation, we studied several models and selected their representative subset. The agile models chosen included eXtreme Programming (XP) and Scrum [4, 15].

We selected these because they were the most widely accepted models [5]. According to [1], they also complement each other. Whereas XP has a definite engineering flavor (pair programming, test driven development, refactoring, continuous integration), Scrum concentrates on the management of software projects [14]. Hence, together they represent a comprehensive agile framework covering both the business and engineering process levels. The result of their synthesis is presented in Figure 1 and Table 1. Each activity in Table 1 is marked with its origin, where *S* stands for Scrum and *XP* stands for eXtreme Programming.

When creating our synthesized model, we discovered that important activities were missing in our synthesized agile model. To ensure its comprehensiveness and that no activity would be missed in this study, we complemented our model with activities from the standard software process model [7]. Our goal was to find out whether they were applicable within the industrial agile evolution and maintenance context. These activities are marked with *HW*, standing for heavyweight.

Using the synthesized *Evolution and Maintenance Process Model*, we then created two questionnaires, one for studying the pre-implementation and one for

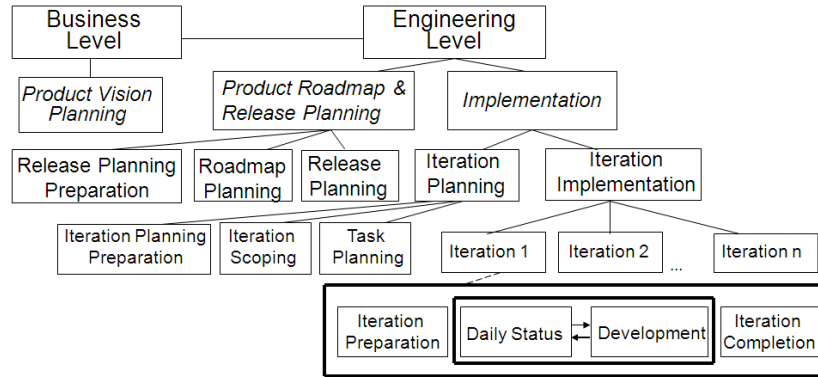


Figure 1. Process phases studied

studying the implementation process phases. The questionnaires were open-ended and semi-structured. They focused on finding out the activities as conducted within the agile evolution and maintenance processes and their status within the organizations studied.

To cover all the phases and their activities, more than 200 questions were created. Due to space restrictions, we cannot list them all. However, most of the questions that were asked followed a somewhat uniform pattern. The pattern was: (1) Do you have this activity? (2) Does it differ from the activity that we describe here? If yes/no, please describe how? (3) Who is responsible for/involved in this activity? Please, state all the roles and their responsibilities; (4) In what form is this activity conducted? (a) by individual? (b) meeting? (c) other form(s)? (5) Is this activity important? If yes/no, please motivate why? (6) Are there any additional activities in this phase that are missing in this model? Could you please describe them?

As a second step, we studied the agile practice by interviewing the representatives of the agile processes in the Canadian companies. The companies were selected according to their relative ease of access, i.e. by the convenience sampling method [13], which means that we considered the companies that agreed to participate in our study. For confidentiality reasons, we do not name them. In Section 2.2, however, we briefly describe them.

Due to the comprehensiveness of this study, we had to make a series of lengthy and time-consuming interviews. In total, we made four interviews, corresponding to about ten hours. In order not to miss anything, we recorded and then documented each interview.

As a next step, we used the results of the interviews to analyze our model from the evolution and maintenance perspective. Here, we realized that the

organizations studied used a set of heavyweight activities. To find out the reasons and to validate the model, we contacted the organizations anew. These reasons are described and motivated in Section 4.4.

2.3 Organizations studied

We interviewed the representatives of the agile development organizations in two Canadian companies applying a mix of Scrum and XP practices.

The first company, denoted as *Organization 1*, develops products ranging from ground stations for satellite radar systems to e-commerce applications. Here, we interviewed a process owner, project manager, and chief architect. We studied one small and one middle-sized agile project evolving data transaction systems. This company took part in both the pre-implementation and implementation study.

The second company, denoted as *Organization 2*, is an IT management consulting firm with 800 employees. It provides IT services and solutions to both private and public sector clients. Here, we interviewed a method owner/project manager regarding the agile process of a small software project. This company took part in both the pre-implementation and implementation study.

3. The process model

This section presents the synthesized *Evolution and Maintenance Process Model*. Its pre-implementation phase is described in Section 3.1 and its implementation phase is described in Section 3.3. In addition, both phases and their activities are listed in Table 1. Due to space restrictions of this paper, we can only briefly describe the phases.

Table 1a. Pre-implementation process phases. S stands for Scrum. XP stands for eXtreme Programming. Bullets (•) and (I) stands for the activities elicited from industry in this study

<u>Product Vision Planning</u>	
A-1.1: Determine product vision (S)	A-1.9.2: Study the system architecture model (S)
A-1.2: Determine expected return on investment (ROI) (S)	A-1.9.3: Identify overall high-level system functionality requirements (S)
A-1.3: Create business case (S)	A-1.9.4: Prioritize identified functionality according to ROI (S)
A-1.4: Conduct business analysis (S)	A-1.9.5: Group the requirements according to product parts, components, other relevant groupings (S)
A-1.5: Create/revise business architecture model (S)	A-1.9.6: Outline future releases (S)
A-1.6: Outline/revise the system architecture model (S)	A-1.9.7: Map prioritized and/or grouped functionality to the releases (S)
A-1.7: Identify quality goals (S)	A-1.9.8: Plan resource allocation (S)
A-1.8: Identify needs for development environment (S)	A-1.10: Record/update high-level requirements specification (HW)
A-1.9: Develop high-level product vision plan (S)	A-1.11: Record product vision plan (HW)
A-1.9.1: Study the business analysis model (S)	• A-1.12: Write contract (I)
<u>Product Roadmap and Release Planning</u>	
<u>Release Preparation</u>	
A-2.1: Schedule product/release planning meeting (S)	A-2.12: Identify new/revise old requirements (HW)
A-2.2: Collect all necessary input to product/release planning (S)	A-2.13: For each requirement:
	A-2.13.1: Assign business value (S) (XP)
	A-2.13.2: Identify risks (S) (XP)
	A-2.13.3: Estimate development effort (e.g. man-days, man-hours) (S) (XP)
	A-2.13.4: Assign priority value (S) (XP)
	A-2.13.5: Record estimates, risks and priority values (HW)
	• A-2.14: Create/update requirements specification (I)
	A-2.15: Prioritize/re-prioritize requirements according to the selected prioritization criteria (S) (XP)
	A-2.16: Re-organize requirement list according to priority value (S) (XP)
	A-2.17: Group requirement list items according to the chosen criteria (S) (XP)
	A-2.18: Scope and outline iterations within release (S) (XP)
	A-2.18.1: Determine number and length of iterations (S) (XP)
	A-2.18.2: Map requirements/requirement groups against identified capacity for iterations (S) (XP)
	A-2.18.3: Determine development feasibility for the iterations and the release (S) (XP)
	A-2.18.4: Calculate development costs (S) (XP)
	• A-2.19: Write acceptance tests (I)
	A-2.20: Confirm release plan (S)
	A-2.21: Record release plan (HW)
	• A-2.22: Create/update backlog, if needed (I)
<u>Product Roadmap Planning</u>	
A-2.3: Study product vision plan (S)	
A-2.4: Identify new/revise old high-level requirements (HW)	
A-2.5: Create requirement list (S) (XP)	
A-2.6: Write/update requirement specification (HW)	
A-2.7: Create product roadmap (S) (XP)	
A-2.7.1: Designate/revise product releases and their contents (S)	
A-2.7.2: Determine product release dates (S)	
A-2.7.3: Determine product release budget (S)	
A-2.8: Record product roadmap (HW)	
<u>Release Planning</u>	
A-2.9: Study product roadmap (S)	
A-2.10: Study requirement list (S)	
A-2.11: Study current version of the requirements specification (HW)	
<u>Iteration Planning</u>	
<u>Iteration Preparation</u>	
• A-3.1: Create Work Definition (I)	A-3.18.5: Identify traceability among the requirements (HW)
A-3.2: Schedule iteration planning meeting (S)	A-3.18.5.1: Identify traceability among the selected iteration requirements (HW)
A-3.3: Collect all necessary input to plan the iteration (S)	A-3.18.5.2: Identify traceability to the already implemented requirements (HW)
<u>Iteration Scoping (Day 1 Planning Part 1)</u>	
• A-3.4: Study Work Definition (I)	A-3.19: Update the requirement specification (HW)
A-3.5: Present current state of business conditions (S)	A-3.20: For each requirement and its analysis model (functional/nonfunctional) (HW)
A-3.6: Present pending features/requirements (S)	A-3.20.1: Identify/revise design goals (HW)
A-3.7: Clarify the meaning and details of the features/requirements (S)	A-3.20.2: Study architecture (HW)
A-3.8: Identify new requirements, if need arises (S)	A-3.20.3: Identify architectural changes, if any (HW)
A-3.9: Identify/revise nonfunctional requirements (HW)	A-3.20.4: Create/modify a design model, if required (HW)
A-3.10: Identify/revise the grouping of the related requirements (HW)	A-3.21: For the requirements planned for the iteration (functional/nonfunctional) (HW)
A-3.11: Identify/revise the prioritization of the requirements (S)	A-3.21.1: Combine their design models (HW)
A-3.12: Select the requirements to be implemented in iteration (S) (XP)	A-3.21.2: Identify quality problems (HW)
A-3.13: Suggest alternative requirements (S) (XP)	A-3.21.3: Attend to the quality problems (HW)
A-3.14: Plan the iteration (S) (XP)	A-3.21.4: Identify traceability among requirements and designs (HW)
A-3.14.1: Define the iteration objective (S)	A-3.22: Agree upon the analysis and design model for the requirements (HW) (XP)
A-3.14.2: For each requirement (S) (XP)	A-3.23: For each requirement, plan its implementation (XP)
A-3.14.2.1: Ensure that the requirement is understood (S) (XP)	A-3.23.1: Identify tasks required for implementing the design model (XP)
A-3.14.2.2: Ensure that the requirement is feasible (S) (XP)	A-3.23.2: Identify dependencies among the tasks (XP)
A-3.14.2.3: Estimate/re-estimate the effort (S) (XP)	A-3.23.3: Sign up for tasks (XP)
A-3.14.2.4: Record the estimated effort for each requirement (S) (XP)	A-3.23.4: Estimate resources required for implementing the task (XP)
A-3.15: Agree upon the iteration plan (S) (XP)	A-3.23.5: Estimate combined resources for the requirement using the resource estimation for each task (XP)
A-3.16: Record the iteration plan (HW)	A-3.24: Record task assignments, including engineers name, estimated effort, other relevant data (XP)
• Task Planning (Day 1 Planning Part 2) results in Task Planning Day 1-3 (I)	A-3.25: Revise the implementation plan (XP)
A-3.17: For each requirement create/modify analysis model(s), e.g. use cases, class diagrams (HW)	A-3.25.1: Revise the task assignments (XP)
A-3.18: For all requirements planned for the iteration (functional/nonfunctional) (HW)	A-3.25.2: Revise dependencies among the tasks and assignments (XP)
A-3.18.1: Combine their analysis models (HW)	A-3.25.3: Revise the estimates (XP)
A-3.18.2: Identify quality problems (inconsistencies, incorrectness incompleteness) (HW)	A-3.25.4: Ensure that task dependencies and estimates are clear (XP)
A-3.18.3: Attend to the quality problems (Goto activity Create/modify analysis model) (HW)	A-3.25.5: Revise the traceability (HW)
A-3.18.4: Identify/revise non-functional requirements (HW)	A-3.26: Identify criteria for accept/reject each requirement implementation (XP)
	A-3.27: Confirm the implementation plan (XP)
	• A-3.28: Write personal/team service level agreement (I)
	A-3.29: Record/update the iteration plan (XP) (HW)

Table 1b. Implementation process phases. S stands for Scrum. XP stands for eXtreme Programming. Bullets (•) and (I) stands for the activities elicited from industry in this study.

<u>Iteration Preparation</u>	
• A-4.1: Setup development environment, if necessary (S) (I)¹ - A-4.1.1: Code management tools (XP) - A-4.1.2: Build machine (XP) - A-4.1.3: Test environment (XP) - A-4.1.4: Back-up infrastructure (XP) - A-4.1.5: Other hardware/software support (XP)	- A-4.2: Establish meeting place and time for daily status meetings (S) - A-4.3: Ensure appropriate facilities for meetings (S) - A-4.3.1: Ensure readily accessible meeting room (S) - A-4.3.2: Equip room with tables (S) - A-4.3.3: Equip room with white board (S) - A-4.3.4: Equip room with speaker phones (S)
¹ The development environment may be set up iteratively as needs grows	
<u>Daily Status (I)</u> ²	
- A-5.1: Ensure that all participants needed are available (S) - A-5.2: Collect all necessary input to track daily progress (S) - A-5.2.1: Collect information about current development status (S) - A-5.2.2: Collect information about the work to be completed before (S) - A-5.2.3: Collect information about any obstacles (S) • A-5.3: Evaluate and assess iteration progress (S) (I)³ - A-5.3.1: Evaluate iteration status (S) - A-5.3.2: Assess the effort spent so far (S) - A-5.3.3: Assess the remaining effort (S) - A-5.4: Identify any further obstacles or project needs (S)	- A-5.5: Update/record collected and assessed information (S) - A-5.6: Allocate and assign resources for removing identified obstacles (S) - A-5.7: Establish follow-up meetings, if needed (S) - A-5.8: Determine the scope of the work to be completed before next status meeting (S) - A-5.9: Revise/confirm commitment to work assignments (S) A-5.10: Update backlog (S) • A-5.10: Update backlog, if needed (I) • A-5.11: Update iteration plan, if needed (S) (I) - A-5.11.1: Revise/update task list and task descriptions, if needed (S) - A-5.11.2: Revise/update task estimates, if needed (S) - A-5.11.3: Revise/update task assignments, if needed (S)
² The daily meeting structure may be impacted for teams larger than 6 developers (e.g. by adding further coordination mechanisms between teams)	
³ The measurement frequency may vary depending on needs	
<u>Development</u>	
• A-6.1: Study work definition, if needed (I) • A-6.2: Write acceptance tests (XP) • A-6.2: Revise existing/write new acceptance tests, if needed (XP) (I)⁴ - A-6.3: Develop product functionality (XP) - A-6.3.1: For the tasks determined in the daily status, hold design session (XP) - A-6.3.1.1: Develop/revise analysis model (XP) - A-6.3.1.2: Develop/revise design model (XP) - A-6.3.2: Implement functionality (XP) - A-6.3.2.1: Pair up in programming pairs (XP) - A-6.3.2.2: Write developers' test cases (unit/unit integration tests) (XP) - A-6.3.2.3: Code functionality (XP) • A-6.3.2.4: Write additional test cases, if needed (I) - A-6.3.2.5: Conduct developers' tests (XP) - A-6.3.2.6: Refactor (XP) - A-6.4: Submit code and test cases for continuous integration (XP) - A-6.5: Conduct continuous integration (XP) - A-6.6: Conduct static testing, if required (HW)	- A-6.6.1: Perform code inspections, if required (HW) - A-6.6.2: Perform static code reviews, if required (HW) - A-6.6.3: Perform static analysis, if required (HW) - A-6.7: Submit for daily build testing (XP) - A-6.8: Conduct daily build testing (XP) - A-6.9: Conduct central integration (HW) - A-6.10: Conduct system tests (HW) • A-6.11: Conduct acceptance tests (XP) (I)⁵ - A-6.12: Measure performance (metrics, empirical controls, burn-down) (S) - A-6.12.1: Update logging of hours of work per task per day (S) (XP) - A-6.12.2: Update test coverage (XP) - A-6.12.3: Update other metrics (HW) • A-6.13: Sign-off personal/team service level agreement (I) • A-6.13.1: Check fulfillment of assignment (I) • A-6.13.2: Complete any remaining tasks/improve quality, if required (I) • A-6.13.3: Approve assignment by final sign-off (I)
^{4,5} Depending on project size and criticality, acceptance tests can be written and conducted by a separate QA role	
<u>Iteration Completion</u>	
- A-7.1: Conduct delivery meeting (S) (XP) - A-7.1.1: Schedule review meeting (S) - A-7.1.2: Give an overview of iteration goal and deliverable (S) - A-7.1.3: Demonstrate deliverable to customer (S) - A-7.1.4: Evaluate deliverable (S) - A-7.1.5: Determine product changes to be considered in next iteration (S) - A-7.2: Make a decision on the continuation of the project (S) - A-7.3: Reprioritize and update requirements list, if needed (S) • A-7.4: Reprioritize and update backlog, if needed (I) - A-7.5: Reformulate the team, if needed (S)	• A-7.6: Assist in managing the release delivery, if needed (HW) (I)⁶ - A-7.7: Conduct iteration retrospective/post-mortem analysis (S) - A-7.7.1: Identify positive and negative trends (S) - A-7.7.2: Determine improvements for next iteration (S) - A-7.8: Compare current project status with business plan (S) - A-7.8.1: Compare ROI (S) - A-7.8.2: Compare budget (S) - A-7.8.3: Compare schedule (S) - A-7.9: Revisit and update the project plan, if needed (S) - A-7.10: Report to business management (S)
⁶ The release requires a separate release management process	

Table 1 outlines the process model that has first been synthesized from the agile methods studied and complemented with some heavyweight activities. These activities are marked with their origin, where S stands for Scrum, XP stands for eXtreme Programming, and HW stands for heavyweight. Their set reflects the status of our model before the industrial study was made.

After having matched the synthesized model to the industrial practice, we noticed some discrepancies between the industrial practice and our model. This, in turn, led us to modify our model to better reflect the industrial state of agile evolution and maintenance practice. Due to space restrictions of this paper, the

modifications are put into the same table, that is, in Table 1. To distinguish from the original activities, they are marked with bullets • before and with an (I) after the activity number.

Finally, we would like point out that the total process structures phases and activities. The goal is to map out what activities are relevant in what stage. However, the process does not impose any specific order of conducting these activities. Depending on the context at hand, these activities may or may not be selected in any order.

3.1. Pre-Implementation Process Phases

As illustrated in Figure 1, the *Pre-Implementation* process phase is executed on both *Business* and *Engineering* levels. It covers the phases and their activities during which one makes various preparations for the next-coming implementation. These are *Product Vision Planning*, and *Product Roadmap and Release Planning*.

Organizations generally have an overall picture of their businesses, core values, strategic goals, mission and product portfolios. In the *Product Vision Planning* phase, this overall picture is evolved into a more detailed product vision. At this phase, one focuses on product goals, overall business and product structure, development environment, and return on investment. *Product Vision Planning* guides work in subsequent planning, decision making, and work [3, 15].

The subsequent phase, *Product Roadmap and Release Planning* consists of three sub-phases: *Release Preparation*, *Product Roadmap Planning* and *Release Planning*. The *Release Preparation* sub-phase involves preparatory activities for the whole phase. Here, one schedules the planning meetings and gathers all the necessary information, including the product vision plan and high-level requirements specification [3, 15].

In the *Product Roadmap and Release Planning* phase, one outlines a high-level roadmap of future product versions. Here, one studies the product vision plan as produced during the *Product Vision Planning* phase, identifies new or revises old high-level requirements, creates a requirements list (a backlog in Scrum or a set of Stories in XP), and updates the requirements specification [3, 15].

3.2. Implementation Process Phases

As illustrated in Figure 1, the *Implementation* process phase is executed on the *Engineering* level. It covers phases and activities during which the actual implementation takes place. These are *Iteration Planning* and *Iteration Implementation*.

Iteration Planning is conducted at the start of each iteration. It involves three phases, *Iteration Planning Preparation*, *Iteration Scoping* and *Task Planning*. Here, the team, product management and other relevant stakeholders meet to plan the implementation work to be conducted in the coming iterations [4, 15].

After having planned the iteration, one steps into the *Iteration Implementation* phase. This phase consists of the sub-phases: *Iteration Preparation*, *Daily Meetings*, *Development* and *Iteration Completion*.

The *Iteration Preparation* is required for the subsequent phases. Its objective is to make initial preparations for starting the consecutive iterations. It involves the setting up of an appropriate environment and ensuring that all the pertinent facilities are in place. According to the agile models studied, it is conducted only once, prior to the development start [3, 15].

The *Daily Status* is a meeting that takes place on a day-to-day basis throughout the whole iteration. The agile models studied suggest that the team gathers briefly (usually, for 15 minutes) to establish project status and to confirm or readjust their commitments, if necessary [3, 15].

The daily meeting is only a quick status meeting. If need arises to discuss additional matters, then one establishes follow-up meetings with the team members concerned. Follow-up meetings usually take place after the daily meeting [3, 15].

In the *Development* sub-phase, the requirements planned for the iteration are implemented and tested. The goal is to deliver an increment of working functionality [4]. Finally, during the *Iteration Completion* sub-phase, one delivers the functionality that has been developed during the iteration, evaluates the iteration, and identifies the improvements to be made in the next iteration [3, 15].

4. Observations Made during our Study

In this study, we noticed some discrepancies and similarities between the industrial practice and our synthesized model. The discrepancies have led us to modify our synthesized model to better reflect the industrial state of practice. As already mentioned, the modifications resulting from the industrial study are marked with bullets next to the activity number and with an *(I)* put after the activity name in Table 1. Below, we list and describe the observations made in Sections 4.1-4.4, respectively.

4.1. Differences Observed in the Pre-Implementation Process Phases

On a general level, our results show that many of the activities as identified in the Pre-Implementation process phases are typically done in the agile projects studied. The differences identified concern (1) contract management, (2) use of ROI, and (3) requirements specification.

4.1.1. Contract Management. Agile models value customer collaboration over contracts [2]. Our results show that the organizations studied negotiate contracts before the *Product Roadmap and Release Planning*

phase. The reason claimed is that some contract items, such as for instance budget, constitute an important prerequisite for carrying out product and release planning. For this reason, we add a new activity for contract writing in the *Product Vision Planning* phase (see A-1.12 *Write Contract* in Table 1a).

4.1.2. Use of ROI. Agile models studied suggest measurement of productivity in ROI in the *Product Vision Planning* phase [15]. The organizations studied however do not make any ROI calculations in this phase. They claim that they lack appropriate tools for doing it at this level. The coarse-grained character of the functionality specified at this level does not allow the organizations to make any meaningful ROI calculations. However, they make ROI calculations in later phases. Despite this, they agree that with appropriate tools, ROI calculations should take place in the *Product Vision Planning* phase.

As a result of this observation, we keep the activities involving the ROI calculation in this phase (see Activity A-1.2 in Table 1a), and do not modify our synthesized model. Instead, we propose that the research community analyses the current ROI tools to find out reasons for why they are claimed to be insufficient.

4.1.3. Requirements Specification in the Product Vision Planning phase. The agile process models are vague about pointing out the phase during which requirements are identified, analyzed and specified. We have observed that the identification of the overall very high-level functionality is already carried out in the *Product Vision Planning* phase (see Activity A-1.9.3). It is documented in a product vision plan called *Operational Concept*, a document corresponding to a high-level system specification. It gives a high-level view of the system needs, its context and operational characteristics.

The *Operational Concept* document is used for various purposes. First, it provides a basis for the developers to relate their changes to the overall system characteristics. Second, it provides a platform for further evolving the system description and requirements specification in the subsequent process phase which is the *Product Roadmap and Release Planning* phase. To address this, we have added the Activity A-2.14: *Create requirement specification* in the *Release Planning* phase into our model (see Table 1a).

4.2. Differences Observed in the Implementation Process Phases

Generally, many of the activities as identified in the *Implementation* process phases are typically done in the agile projects studied. The differences identified concern (1) iteration planning practice, (2)

personal/team *Service Level Agreements*, (3) Duration of task planning, (4) timing of the daily meetings, (5) portfolio of status meetings, (6) project scalability, (7) measurement guidelines and frequency, (8) pair programming practice, (9) refactoring practice, (10) test-programming practice, (11) release management practice, (12) number of iteration review meetings, and (13) placement of post-mortem analysis.

4.2.1. Iteration Planning Practice. Scrum suggests that the planning of iterations be prepared by scheduling meetings and by gathering the necessary information [15]. We have found that the iteration preparation activity involves a study of a document called *Work Definition*, produced prior to the iteration planning. It functions as a driving wheel of the iteration planning sessions by providing an overview of the iteration plan. We believe that this is a relevant aspect to consider in addition to the two already existing activities of the *Iteration Preparation* phase (see activities A-3.2 and A-3.3 in Figure 2). Creating such a document also results in new input to the *Iteration Scoping* phase. Hence, we add two new activities. These are A-3.1 *Create Work Definition* in the *Iteration Preparation* phase (see Table 1a) and A-3.4 *Study Work Definition* in the *Iteration Scoping* phase (see Table 1a).

4.2.2. Personal/Team Service Level Agreements. The agile models do not suggest contract writing neither on the developers nor on the team level. In *Organization 1*, we have however observed that developers write under contracts that are official agreements between developers and a chief system engineer.

At the beginning of an iteration, developers signs off the undertaking of the tasks assigned to them. These undertakings are then combined into a *Team Service Level Agreement*. The reason is that the responsibility for delivering the iteration results should be shared by the whole team.

At the end of the iteration, during the activity called “*Sign-Off*” in the *Iteration Completion* phase, the deliverables provided by the developers and teams are measured against these agreements. Here, the architect checks whether the tasks promised in the *Team SLA* have been fulfilled, i.e. that the code, tests, design documentation, and the quality goals have been fulfilled.

Organization 1 claims that the sign-off activity is crucial. They have found that developers often implement only 80% of a particular task and forget about the remaining 20%. Before they introduced the “*Sign-Off*” activity, this happened all the time. Introducing it at the end of the iteration has therefore become an effective control of the task fulfillment

ensuring that the team delivers according to their initial commitment.

If the “*Sign-Off*” is not approved in *Organization 1*, then the unfulfilled tasks must be addressed directly, unless they are large and require considerable re-work. The plan allows two days after the “*Sign-Off*” activity to finish any incomplete or unapproved tasks.

This practice is considered as a necessary quality assurance activity. For this reason, we have added it to our model by introducing two activities: (1) *A-3.28 Write personal/team service level agreement* at the end of the *Iteration Planning* phase (see Table 1a) and (2) *A-6.13 Sign-off personal/team service level agreement* at the end of the *Development* phase (see Table 1b).

4.2.3. Duration of Task Planning. Agile models suggest that the *Task Planning* takes place during the first day of the iteration [4]. We have however observed that it takes longer than one day in the organizations studied. Within middle-sized projects in *Organization 1*, planning the tasks could sometimes take up to three days. Its duration depends on the complexity and criticality of the tasks and the skills of the engineers. This observation has made us extend the number of days dedicated to *Task Planning* from one day to 1-3 days instead (see Table 1a). We also propose that the models revise their suggestions for the duration length of this activity.

4.2.4. Timing of the daily meetings. The agile models prescribe status meetings in the morning [4]. Our study shows evidence that this timing was not effective. In the organizations studied, the daily meetings are kept in the afternoon. The reason is a substantial improvement of meeting effectiveness.

The organizations studied have found that it is difficult to have an effective and informative meeting in the morning because people tend to come to work at different times. There is also a risk that issues encountered during the previous day get forgotten.

Just because the timing aspect is not explicitly covered in our model, we do not modify it. However, in accordance with our results, we state that it is critical to find an appropriate time when all team members can contribute to the meeting effectiveness.

4.2.5. Portfolio of status meetings. The agile models only suggest daily status meetings within the team and between teams (*Scrum of Scrums*) [15]. We have however observed that the organizations studied have complementary management and corporate-level status meetings conducted on a weekly basis. On those meetings, one presents an overall organizational view of project status and discusses various organizational and technical issues that might impact the project.

These meetings are kept irrespective of whether the teams are small or large.

The corporate-level status meetings in the organizations studied are driven by the organizational standards and information needs, rather than by the agile process. For this reason, we regard them as optional and therefore, we do not modify our process model.

4.2.6. Coverage of project scalability. The agile models studied provide relatively little or different guidelines for scalability. XP allows teams to consist of 20 developers [4], while Scrum suggests team sizes of 7 +/-2 people [15]. Scrum however proposes *Scrum of Scrums* meetings to help coordinate the work between several teams [15].

In accordance with Scrum and the results reported in [6], our findings also indicate that the size of a project affects both the structure of the teams, procedures and the scheduling of the daily meetings. According to one of our interviewees, additional coordination mechanisms are required for all projects that are larger than six developers. As a result of these findings, we add a note clarifying that scalability might impact the meeting structures and that supplementary coordination mechanisms may be needed (see *Note 2* in the *Daily Status* phase in Table 1b).

4.2.7. Measurement guidelines and frequency. The organizations studied do not use burn-down charts for measuring their daily progress as prescribed by Scrum or story points as prescribed by XP [4]. The management regards the agile guidelines as one-dimensional, insufficient and difficult for making predictions. However, our interviewees agreed that they still provide an attractive and simple useful way to start measuring the status within in the team.

Regarding the measurement frequency, the agile models studied suggest measurement of the progress on a daily basis [3, 15]. However, measurement on a day-to-day basis is regarded to be too granular and too time consuming by the organizations studied. According to them, measurement on a weekly basis provides more useful figures about the progress in their projects.

Regarding the measurement aspects in the agile environment, we do not take any standpoint. We feel that measurement in an agile context is an unexplored issue and needs be further elaborated on. Hence, we do not make any modifications to our model.

4.2.8. Pair programming practice. Pair programming is prescribed by XP [4], but not by Scrum as it focuses on the management activities. However, pair programming is not conducted in the organizations studied. It is regarded to be too expensive and the programmers are not interested in doing it. We do not modify our model due to this

observation. We believe that it should be an option and not a must, as far as other forms of achieving good communication, collaboration and shared project responsibility are practiced. However, we agree that this practice constitutes an effective tool when managing evolution and maintenance changes.

4.2.9. Refactoring practice. XP strongly advocates refactoring [3, 4]. Despite this, refactoring is not promoted in the organizations studied. If used, then only on an ad hoc manner, depending on the team and each programmer's personal preference.

Management and some senior engineers are the main reason for this. In their opinion, refactoring is an excuse for old sins, sins providing the evidence that developers have not thought through their work upfront good enough. The observation of this difference does not lead to any changes in our model. We believe that refactoring is an important maintainability aspect that should be seriously promoted within the industry.

4.2.10. Test-first programming practice. Test-first programming is conducted to a limited extent, only in *Organization 1*. Although the developers have guidelines for test-first programming, they do not fully follow them. They only write test cases upfront for known and well-defined classes. They then write the remaining tests either during or after the coding.

The interviewee in *Organization 1* claims that test-first programming is technically difficult and not always applicable. For instance, some test cases (i.e., white-box test cases) cannot be written in advance. Due to the fact that test-first programming requires a complete change of the programming mindset, it is also difficult to implement straightforwardly. It forces the developers to reorder the development activities, and write tests as part of the design process. Still, however, the interviewee recognizes many benefits of test-first development and claims that they strive towards achieving this practice as much as possible.

This finding has also been confirmed in [10]. Hence, we add the step of writing additional tests during and after coding into our model (see Activity A-6.3.2.4 *Write additional test cases* in Figure 3).

4.2.11. Release management practice. Both organizations studied conduct release management. Their processes however differ somewhat depending on the project size. In *Organization 2*, the process is performed by the development team. In *Organization 1*, the scenario varies depending on the project size and the customer's technical capability of managing the release process.

In case of small projects where the customer does not have the capacity or skills to manage the release, the development team manages the entire release process. They first assist the customers in the

acceptance testing. They then package the system and install it on the customer's site.

In case of medium-sized and large projects, where the customers have the capacity or skills to manage the release, they manage the release process by themselves. They either take full responsibility for the whole process or part of it. The development team only provides some assistance. For instance, it packages the software system and submits it into a known location on the customer site. The customer then performs acceptance testing and installation on his own.

Regarding the release management roles, *Organization 1* has defined two roles dedicated to release management whereas *Organization 2* has only one role (*Release Manager*). The roles of *Organization 1* are *Primary Release Captain* and a *Secondary Release Backup*. These roles are rotated around the team members; the *Secondary Release Backup* will become the *Primary Release Captain* in the next iteration, and a new *Secondary Release Backup* gets chosen in the team.

The agile models studied do not provide any explicit guidelines for the activities involved in release management. As described above, the development teams have a release process in place to ensure that the outcome of the iteration is delivered. To point out that a set of release management process activities are required, we add a note that clarifies this (see *Note 6* for Activity A-7.6 in Table 1b).

A.2.12. Number of iteration review meetings. Both organizations have iteration review meetings during which the product is demonstrated and reviewed by the team and the customer. *Organization 2* has only one meeting where the project manager, the team and the customer discuss the deliverable and any possible changes in the next iteration. *Organization 1*, on the other hand, has three separate meetings. These are:

- *Meeting 1:* It is a quick one hour meeting where the deliverable is demonstrated internally within the team to check whether the iteration goals have been fulfilled and to identify improvements in the next iteration.
- *Meeting 2:* The developers walk through the deliverable to demonstrate a tangible proof of the iteration results to the customer.
- *Meeting 3:* The architect demonstrates the deliverable to the QA group to get their final approval.

The agile models studied prescribe two review meetings at the end of the iteration [3, 15]. Both organizations studied conduct reviews according to the agile guidelines. As described above, the only difference is the number of meetings. Because the purpose is the same, we do not modify our model.

A-4.13. Placement of Post-Mortem Analysis.

Both organizations studied conduct post-mortem analyses. However, they conduct them at different formality levels and at different points in time. *Organization 2* has a brief post-mortem analysis at the same time as the above-mentioned delivery meeting. *Organization 1*, on the other hand, generally conducts an informal post-mortem analysis in the initial phase of the *Implementation* phase, that is, in the *Iteration Planning* phase (see Figure 1a). This observation does not lead to any changes in our model.

4.3. Differences Observed both in the Pre-Implementation and Implementation Phases

In this section, we report on the differences common to both the *Pre-Implementation* and *Implementation* process phases. These are (1) use of backlog, (2) points in time when acceptance tests are written, and (3) roles involved in acceptance testing.

4.3.1. Use of Backlog. Scrum defines the backlog as a prioritized list of requirements that drives the release planning, iteration planning and implementation. Scrum also suggests that it be maintained by a product owner [15]. In the organizations studied, the objective of the backlog differs from the one defined in Scrum. It is maintained by the team and serves only as a complementary list of work to be done within the team. Other documents are used for planning and organizing the iteration work, such as the *Operational Concept* and *Work Definition*.

The *Work Definition* document is claimed to be the primary driving wheel of future iterations. It plans the next-coming iterations and assigns requirements to them. This observation does not lead us to make any observable changes in our model. We believe that one should be free to decide on the media used for planning and managing the iterations. We however clarify that a backlog can be used as a complimentary driving wheel of iterations. We do it by:

- adding the Activity A-2.22 *Create/update backlog, if needed* in the *Release Planning* phase in Table 1 a,
- splitting the original activity A-5.10 *Update backlog* into A-5-10 *Update backlog, if needed* and A-5.11 *Update iteration plan, if needed* (see the *Daily Status* phase in Table 1b),
- adding the Activity A-7.4 *Reprioritize and update backlog, if needed* in the *Iteration Completion* phase to further reflect the use of the concept of backlog in the industry.

4.3.2. Point in time when acceptance tests are written. XP suggests that tests be written early, preferably before the implementation starts [4]. However, it does not provide any clear guidelines

about the exact point in time when the acceptance tests should be written. Our results show that writing of some of the acceptance tests already starts during the *Pre-Implementation* phase (see *Product Roadmap and Release Planning* phase in Figure 1). As a result of it we infuse the following changes into our model:

- add a new activity (A-2.19 *Write acceptance tests*) in the *Product Roadmap and Release Planning* phase (see Table 1a),
- reword the activity A-6.2 *Write acceptance tests* to A-6.2 *Revise existing/write new acceptance tests, if needed* instead.

By doing so, we adapt our model to two scenarios: (1) the acceptance tests are written before the implementation phase which is the case of larger evolution and maintenance projects, and (2) the acceptance tests are written at the beginning of the implementation phase, which is mainly the case of small innovative projects.

4.3.3. Roles involved in acceptance testing. The agile models studied suggest that acceptance tests be written and performed by the customer together with the developers [4]. Our observations show that in larger projects, acceptance testing is conducted by a separate QA team. Essentially, the acceptance testing activities in larger projects remain the same, however, there may be different roles conducting them. For this reason, we do not modify our model. However, we clarify that a separate QA team may be introduced in larger and more critical projects (see *Notes 4* and *5* in the *Development* phase in Table 1b).

4.4. Other Observations

We have found that several of the activities that we have added to the synthesized model from the standard software development model [7] are applied in the organizations studied. Many of these activities are typical for heavyweight processes. They mainly concern the pre-implementation activities such as requirements elicitation, analysis and specification, architectural analysis and design, documentation practice, guidelines regarding scalability, risk management and thorough task planning.

This observation has made us contact the interviewees anew to find out the reasons. Our interviewees claim that the degree of agility of the pre-implementation phases varies among projects. It depends on the phase in the product lifecycle, project type, its size, criticality, innovative character, degree of uncertainty, risk taking, permission from stakeholders' side to take risk, and budget.

The majority of their evolution and maintenance projects follow a more traditional pre-implementation approach. The small, innovative, creative and totally

new projects, on the other hand, follow a more agile approach. In these projects, one has difficulties to conduct detailed planning in advance. Hence, one mainly outlines product vision and high level requirements within the pre-implementation phases and allows the rest to be resolved within the later phases.

Finally, our interviewees claim that in the context of evolution and maintenance projects, one must still plan upfront. According to them, good planning upfront in the *Pre-Implementation* phases allows more agility in the *Implementation* phase. The key issue is to set up an instructive plan and then let the teams decide on the implementation process approach. With good and thorough plans, any degree of agility works well as long as long as one keeps control of the product vision, goals and fulfillment of these goals.

5. Final remarks

The synthesized evolution and maintenance process model presented in this paper is based on a subset of the existing agile process models. It also reflects the state of practice within the two software organizations studied. Our results show that the organizations studied have adopted an agile approach within evolution and maintenance of their systems. However, the degree of agility varies across the phases. It is higher in the *Implementation* phase and lower in the *Pre-Implementation* phase. This observation makes us conclude that some agile aspects may be relevant in the context of software evolution and maintenance. More studies however need to be made to confirm the premises presented herein.

6. References

- [1] Abrahamsson P. et al., Agile Software Development Methods: Review and Analysis. VTT Electronics/ Oulu University, 2002.
- [2] Beck K., Extreme Programming Explained: Embrace Change. Addison-Wesley, 2000.
- [3] Beck K., Test Driven Development: By Example. Pearson Education, Boston, MA, 2003.
- [4] Beck K., Extreme Programming Explained: Embrace Change. 2nd Ed. Addison-Wesley, 2004.
- [5] Charette R., The Decision is in: Agile vs Heavy Methodologies (2001). Cutter Consortium, Vol. 2 (19) <http://www.cutter.com/content/project/fulltext/updates/2001/epmu0119.html>. Accessed in October 2007.
- [6] Cohen D. et al., Agile Software Development: a DACS State-of-the-Art Report. Fraunhofer Center for Experimental Software Engineering/University of Maryland, 2003.
- [7] IEEE Standards, IEEE Std 12207.0-1996 Software Life Cycle Processes. IEEE Inc. New York, NY, 1998.
- [8] Jeffries R., Anderson A. and Hendrickson C., eXtreme Programming Installed. Addison-Wesley, 2000..
- [9] Kajko-Mattsson M. and Yulong F., Outlining a Model of a Release Management Process. Journal of Design & Process Science: Trans. of the SDPS, Vol. 9 (4), 2005, pp. 13-25.
- [10] Kajko-Mattsson M., Björnsson T., Outlining Developers' Testing Process Model. Proc. of Euromicro Conference on Software Engineering and Advanced Applications, 2007
- [11] Kaner et al., Testing Computer Software. John Wiley & Sons, 2001.
- [12] Merriam-Webster, Merriam-Webster Dictionary Online [Online]. URL: <http://www.m-w.com/dictionary/backlog> . Accessed December 2007.
- [13] Robson C., Real World Research. Blackwell Publishing, 2002.
- [14] Schwaber K., Scrum Development Process. Proc. of Conference on Object-Oriented Programming, Systems, Languages and Applications, 1995.
- [15] ScrumMethodology, Scrum Methodology: Incremental, Iterative Software Development from Agile Processes. Rev. 0.9. Advanced Development Methods Inc., 2003.